Software Project Management

CSE 421 | Session 2020-21 | Final exam

Model study answers. Every listed option is addressed. Question titles are paraphrased; exact wording and original diagrams are in the source pages appended to this pack.

The final paper totals 50 marks over 2 hours. Its COCOMO constants differ from common textbook tables; the worked answer uses the printed constants. Course reading links are now available below. The course site does not define every term in this newer paper; those answers retain explicit interpretation notes.

Quick revision

PDCA: Plan -> Do -> Check -> Act.

SMART: Specific, Measurable, Achievable, Relevant, Time-bound (wording varies by course).

Basic COCOMO: effort E = a x KLOC^b person-months.

Schedule T = c x E^d months. Average staff = E/T.

Use the supplied exam constants: Organic (2,1,2,1), Semi-detached (3,2,2,1), Embedded (4,3,2,1).

W5HH: Why, What, When, Who, Where, How, How much.

Risk exposure is often modeled as probability x impact, using compatible units.

1(a) | Internal stakeholders

Internal stakeholders belong to the organization undertaking the project and affect or are affected by it. Examples include the project sponsor, project manager, developers, testers, operations staff and department managers. Distinguish them from external clients, regulators and suppliers.

Reading: Related course reading · Introduction - https://mhrumi.github.io/Software-project-management/01-introduction.html

Revision answer: Stakeholders inside the developing organization, such as the manager and development team.

1(b) | Generic software

Generic software is developed for a broader market rather than one specified customer. The supplier typically controls product requirements and sells or distributes it to many users. Examples include a word processor, a general accounting package or an off-the-shelf photo editor. Bespoke software is commissioned for a particular customer's needs.

Reading: Related course reading · Introduction - https://mhrumi.github.io/Software-project-management/01-introduction.html

Revision answer: A product made for many customers rather than one bespoke client.

1(c) | Code-Like-Hell Programming

This classical mistake means rushing into intensive coding without adequate requirements, design, planning or disciplined control. It can appear productive initially but creates integration failures, rework and unpredictable delivery. Avoid it with proportionate planning, clear priorities, incremental delivery and engineering discipline.

Reading: Related course reading · Project approach - https://mhrumi.github.io/Software-project-management/04-project-approach.html

Revision answer: Unplanned frantic coding creates rework and schedule risk.

1(d) | Short-Changed Upstream Activities

This mistake underinvests in early work such as feasibility, requirements, architecture and planning to appear to save time. Ambiguities and design weaknesses then become expensive downstream defects. Invest enough in early clarification and review without demanding perfect upfront knowledge.

Reading: Related course reading · Introduction - https://mhrumi.github.io/Software-project-management/01-introduction.html

Revision answer: Skipping essential early analysis shifts greater cost into later rework.

1(e) | Project scope

Scope defines the work and deliverables included in a project, together with boundaries, exclusions and acceptance conditions. Example: a library system includes catalog search and lending, but excludes payroll. A clear scope supports estimates and controls scope creep through an agreed change process.

Reading: Related course reading · Step Wise planning - https://mhrumi.github.io/Software-project-management/03-project-planning.html

Revision answer: Scope states what the project will and will not deliver.

1(f) | PDCA loop with a software example

Plan: identify a problem and define a measurable improvement, such as reducing deployment failures. Do: pilot automated checks in one team. Check: compare failure data with the baseline. Act: standardize the successful change or revise the approach and repeat. PDCA is an iterative improvement loop, not a one-time checklist.

Reading: Related course reading · Software quality - https://mhrumi.github.io/Software-project-management/13-software-quality.html

Revision answer: Plan an improvement, try it, assess evidence and adopt or revise it.

1(g) | SMART objectives

A common expansion is Specific, Measurable, Achievable, Relevant and Time-bound. Example: reduce the median checkout response time below two seconds under a defined test load by the end of the next release. Some course texts use Assignable or Realistic in place of particular words; use the lecture's variant if supplied.

Reading: Related course reading · Step Wise planning - https://mhrumi.github.io/Software-project-management/03-project-planning.html

Revision answer: Specific, Measurable, Achievable, Relevant, Time-bound.

2(a) | Software crisis: reasons and example

The software crisis describes persistent difficulty delivering large software systems on time, within budget and with acceptable quality. Causes include growing complexity, unclear or changing requirements, weak estimates, poor communication, inadequate methods and insufficient testing. Example: a payroll project estimated as simple expands to complex tax and integration requirements, causing delays and incorrect payments. This is an illustrative example, not a claim about a named historical project.

Reading: Related course reading · Introduction - https://mhrumi.github.io/Software-project-management/01-introduction.html

Revision answer: Complexity and weak requirements, planning and engineering cause cost, schedule and quality failures.

2(b) | Five project-management process groups

Initiating: authorize a project and identify stakeholders, such as approving a student portal charter. Planning: define scope, schedule, resources, quality and risk plans. Executing: coordinate people to build and test deliverables. Monitoring and controlling: compare actual progress with the plan and manage changes, defects and risks. Closing: obtain acceptance, hand over operations, close contracts and record lessons. These groups can recur or overlap; they are not necessarily five sequential SDLC phases.

Reading: Related course reading · Introduction - https://mhrumi.github.io/Software-project-management/01-introduction.html

Revision answer: Initiate, plan, execute, monitor/control and close.

2(c) | CI/CD pipeline

Continuous integration frequently merges small changes into a shared repository, triggering automated build and test checks. Continuous delivery keeps tested changes deployable, often with a manual production approval. Continuous deployment automatically releases changes that pass the defined gates. Example pipeline: commit -> build -> unit/integration/security checks -> staging -> acceptance gates -> release -> monitoring. Failed checks block promotion and provide feedback.

Revision answer: Automate integration, verification and safe release; delivery and deployment differ in production approval.

2(d) | Risk versus opportunity

A risk is an uncertain event or condition that can affect objectives; many project frameworks include both threats and opportunities. In common classroom contrast, risk means a downside threat and opportunity means a beneficial uncertainty. Real-life threat: rain may delay an event; opportunity: unexpectedly good weather may increase turnout. Software threat: a dependency may fail; opportunity: a reusable component may shorten delivery. Manage threats and opportunities deliberately rather than assuming uncertainty is always negative.

Reading: Related course reading · Risk management - https://mhrumi.github.io/Software-project-management/07-risk-management.html

Revision answer: Threats can harm objectives; opportunities can improve them; both involve uncertainty.

2(e) | Person-month, Brooks's law and cost drivers

A person-month measures effort equivalent to one person working for one month under a stated convention; it is not the same as elapsed duration. Brooks's law warns that adding people to a late software project can make it later because onboarding and communication consume time and some work cannot be parallelized. COCOMO cost drivers are factors such as required reliability, complexity, team capability and tool support that adjust effort in intermediate/detailed models. Basic COCOMO uses size and mode coefficients; do not silently add an unspecified effort multiplier.

Reading: Related course reading · Estimation models - https://mhrumi.github.io/Software-project-management/05b-estimation-models.html

Revision answer: Effort differs from duration; added staff have overhead; cost drivers adjust model effort.

2(f) | Manage development of a YouTube-like platform

Initiate: establish the audience, business goal and feasibility. Plan: define an MVP such as accounts, upload, playback and search; estimate effort/cost, build a WBS, schedule milestones and identify scalability, moderation and privacy risks. Execute: build incrementally with architecture, implementation, automated tests and deployment work. Monitor/control: track cost, schedule, playback performance, defect rates and scope changes; respond to risks. Close/handover: obtain acceptance, document operations and recovery, train support and record lessons. Treat future features as planned increments rather than unlimited initial scope.

Reading: Related course reading · Introduction - https://mhrumi.github.io/Software-project-management/01-introduction.html

Revision answer: Define and estimate an MVP, deliver iteratively, monitor quality/risk and hand over operations.

3(a) | W5HH principle with examples

Why is the system being developed? To reduce payroll errors. What will be done? Employee records and salary processing. When will it be done? Agreed milestone dates. Who is responsible? Named owners for requirements, implementation and testing. Where are they organizationally located? Finance, engineering and operations roles. How will the job be done? Chosen process, architecture and tools. How much resource is needed? Estimated effort, staffing and budget. These seven questions align purpose, responsibility and execution.

Reading: Related course reading · Estimation models - https://mhrumi.github.io/Software-project-management/05b-estimation-models.html

Revision answer: Why, What, When, Who, Where, How and How much.

3(b) | COCOMO: payroll versus flight control

Use Basic COCOMO E=a*KLOC^b, T=c*E^d and average people=E/T. Classify the small simple payroll as Organic and the complex real-time hardware-dependent flight-control system as Embedded. Both use KLOC=40.

Payroll: printed Organic constants a=2,b=1,c=2,d=1. E=2*40^1=80 person-months. T=2*80^1=160 months. Average staff=80/160=0.5 people.

Flight control: printed Embedded constants a=4,b=3,c=2,d=1. E=4*40^3=4*64,000=256,000 person-months. T=2*256,000=512,000 months. Average staff=256,000/512,000=0.5 people.

Flight-control effort and duration are each 3,200 times payroll under this supplied table. These extreme durations and fractional average staffing expose the unusual constants; they are algebraic model outputs, not credible staffing recommendations. Do not replace the paper's coefficients with familiar textbook values without saying so. Fractional average effort can mean part-time allocation; it is not a literal half person.

Reading: Related course reading · Estimation models - https://mhrumi.github.io/Software-project-management/05b-estimation-models.html

Revision answer: Using the printed constants: payroll E=80, T=160; flight E=256000, T=512000; both E/T=0.5.

3(c) | Risk in the Slack surge scenario

Treat the scenario as the exam's hypothetical account, not a verified incident history. The direct issue is a technical/operational capacity and availability risk: demand exceeds the service's ability to operate reliably. It can be called a performance or scalability risk depending on the course taxonomy. Reduce it with realistic load/stress testing, capacity forecasting, scaling plans, queueing and graceful degradation, monitoring, redundancy and an incident/recovery plan. Contingency capacity prepares for spikes rather than merely responding after failure.

Reading: Related course reading · Risk management - https://mhrumi.github.io/Software-project-management/07-risk-management.html

Revision answer: A capacity/scalability and availability risk, addressed through testing, resilient design and planning.

4(a) | Continuous Process Improvement

CPI is ongoing, evidence-based refinement of how work is performed to improve quality, efficiency or reliability. Measure a baseline, identify a bottleneck, try a change and inspect results. Example: use defect data to improve reviews, then verify whether escaped defects decrease.

Reading: Related course reading · Software quality - https://mhrumi.github.io/Software-project-management/13-software-quality.html

Revision answer: Repeated measurable improvement of the way work is done.

4(b) | MoSCoW prioritization

Must have: essential for a viable release, such as correct payment calculation. Should have: important but can temporarily be worked around, such as exportable reports. Could have: desirable if capacity permits, such as theme customization. Won't have this time: explicitly excluded from the current scope, such as advanced forecasting. Agree criteria and prevent every feature being labeled Must.

Reading: Related course reading · Risk management - https://mhrumi.github.io/Software-project-management/07-risk-management.html

Revision answer: Must, Should, Could, Won't have this time.

4(c) | Two reasons for risk management

First, it identifies threats early so the team can reduce their likelihood or impact before they cause major rework. Second, it prepares contingency actions and resource decisions, improving the predictability of cost, schedule and quality. It does not eliminate every uncertainty.

Reading: Related course reading · Risk management - https://mhrumi.github.io/Software-project-management/07-risk-management.html

Revision answer: Reduce preventable losses and prepare responses to uncertainty.

4(d) | Blended approach to risk management

A reasonable interpretation combines proactive prevention with reactive contingency and recovery. For data loss, improve storage reliability and access controls proactively, while retaining tested backups and an incident response plan for failures. The term can be course-specific; if your lecture defines a particular blend or framework, use its definition.

Reading: Related course reading · Risk management - https://mhrumi.github.io/Software-project-management/07-risk-management.html

Revision answer: Combine prevention with prepared response and recovery; check course-specific terminology.

4(e) | Purpose of Jenkins

Jenkins is an automation server commonly used to run CI/CD pipelines. It can trigger builds and tests after commits, coordinate checks and automate deployment steps through configured jobs and plugins. It supports the workflow; it does not itself guarantee software quality.

Revision answer: Automate builds, tests and delivery workflows.

4(f) | Software Process Improvement

SPI systematically evaluates and changes software development/maintenance processes to improve outcomes such as product quality, delivery predictability and efficiency. Examples include adding peer review or improving defect tracking and checking whether results improve. Frameworks may guide assessment, but outcomes matter more than documentation alone.

Reading: Related course reading · Software quality - https://mhrumi.github.io/Software-project-management/13-software-quality.html

Revision answer: Improve software processes through assessment, change and measurement.

4(g) | Risk mitigation

Mitigation takes actions to lower a threat's probability and/or impact. Example: automated regression tests reduce the chance of release defects, and rollback capability reduces their operational impact. Strict taxonomies distinguish mitigation from avoidance, transfer and acceptance; some exam wording uses mitigation broadly for all responses.

Reading: Related course reading · Risk management - https://mhrumi.github.io/Software-project-management/07-risk-management.html

Revision answer: Reduce the probability or impact of an identified threat.

5(a) | Key clauses of ISO 9001

For the ISO 9001:2015 structure commonly taught with this exam, clauses are: 1 Scope; 2 Normative references; 3 Terms and definitions; 4 Context of the organization; 5 Leadership; 6 Planning; 7 Support; 8 Operation; 9 Performance evaluation; 10 Improvement. Clauses 4-10 contain the main quality-management requirements. The question does not specify an edition, so identify the 2015 assumption; standards can change and this is a historical exam answer rather than a claim about the latest edition.

Reading: Related course reading · Software quality - https://mhrumi.github.io/Software-project-management/13-software-quality.html

Revision answer: For ISO 9001:2015, main requirement clauses are Context, Leadership, Planning, Support, Operation, Evaluation and Improvement.

5(b) | Why WBS matters for scope

A work breakdown structure decomposes the full project scope into manageable deliverables and work packages. It reveals omissions, supports estimates and responsibility assignment, and provides a baseline for change control. Apply the 100% rule: child elements collectively cover the parent scope without double-counting. A WBS is not by itself a timeline or dependency network.

Reading: Related course reading · Step Wise planning - https://mhrumi.github.io/Software-project-management/03-project-planning.html

Revision answer: WBS makes scope complete, estimable and assignable.

5(c) | Project manager's role in planning and scope

The manager coordinates stakeholders to clarify goals, requirements, exclusions and acceptance criteria; creates or coordinates the scope baseline and WBS; estimates resources, cost and schedule; assigns responsibilities; and establishes risk, communication and change-control plans. Example: for a portal, agree that registration is included but hostel allocation is deferred, then assess any request to add it against time and cost.

Reading: Related course reading · Step Wise planning - https://mhrumi.github.io/Software-project-management/03-project-planning.html

Revision answer: Align expectations, establish the plan and control scope changes.

5(d) | Proactive versus reactive risk strategies

Proactive management identifies and analyzes possible problems before they occur and implements preventive measures, such as load-testing before registration day. Reactive management responds after a problem occurs, such as restoring service after an outage. Prepared contingency responses are planned proactively but executed when triggered. Good management combines prevention with recovery readiness.

Reading: Related course reading · Risk management - https://mhrumi.github.io/Software-project-management/07-risk-management.html

Revision answer: Proactive reduces risks beforehand; reactive addresses their realized consequences.

5(e) | Software versus traditional project management

Both manage scope, time, cost, people, quality and risk. Software adds distinctive challenges: intangible deliverables, high requirements volatility, complex integration, difficult progress visibility, knowledge-intensive work and cheap replication after development. Iterative delivery and automated tests can give early feedback. Avoid claiming traditional projects never change or never use iterative methods; the differences are matters of degree and domain.

Reading: Related course reading · Software quality - https://mhrumi.github.io/Software-project-management/13-software-quality.html

Revision answer: Shared management foundations, with greater software-specific uncertainty and intangible progress.

5(f) | Major causes of software project failure

Common contributors include unclear goals, unstable/uncontrolled scope, unrealistic estimates, weak stakeholder engagement, poor communication, missing skills/resources, architectural/integration problems, inadequate testing and unmanaged risk. These reinforce one another: rushed planning creates an impossible schedule, which encourages skipped checks and rework. Define failure against agreed goals rather than assuming delay is the only criterion.

Reading: Related course reading · Software quality - https://mhrumi.github.io/Software-project-management/13-software-quality.html

Revision answer: Requirements, planning, communication, engineering quality and risk failures often interact.

6(a) | WBS for an e-commerce website

Level 1: complete e-commerce website. Level 2: major deliverables such as project management, storefront, account/identity, catalog/search, cart/checkout, order management, platform/integration, quality/security and release/handover. Level 3: decompose checkout into address entry, delivery choice, payment integration and confirmation. Level 4: create estimable work packages with owner, acceptance criteria and boundaries, such as implementing and testing the payment callback. Include cross-cutting work explicitly, cover 100% of the agreed scope and avoid duplicated work. Add dependencies and dates in a schedule after defining the WBS.

Reading: Related course reading · Step Wise planning - https://mhrumi.github.io/Software-project-management/03-project-planning.html

Revision answer: Decompose the whole product into deliverables, components and manageable work packages.

6(b) | CMM level and all higher maturity levels

Using the classic Software CMM taxonomy, organization-wide standardized and documented SDLC processes plus training point to Level 3: Defined. At Level 3, processes are established and tailored consistently across projects. Level 4: Managed adds quantitative measurement and statistical control of process/product performance. Level 5: Optimizing emphasizes continuous process improvement, defect prevention and innovation using quantitative evidence. Context: Level 1 is Initial and Level 2 is Repeatable. Do not silently mix these names with CMMI, whose level 2 and level 4 labels differ. The scenario suggests a level; a formal appraisal would need more evidence.

Reading: Related course reading · Software quality - https://mhrumi.github.io/Software-project-management/13-software-quality.html

Revision answer: Likely CMM Level 3 Defined; then Level 4 Managed and Level 5 Optimizing.

6(c) | Four risk-response strategies for the data-loss scenario

Avoid: remove the risky activity, for example do not store unnecessary sensitive customer data. Reduce/mitigate: use reliable storage to lower likelihood, and tested daily backups to lower impact; these are two dimensions of the same response category. Transfer/share: buy cyber-insurance for specified financial consequences; it does not remove technical responsibility. Accept: knowingly tolerate small residual corruption risks with monitoring and a defined response. The question asks for four strategies, but its explicit reliability and backup examples both represent reduction. The avoidance example is added to complete the standard four-strategy set.

Reading: Related course reading · Risk management - https://mhrumi.github.io/Software-project-management/07-risk-management.html

Revision answer: Avoid, reduce, transfer and accept; reliability and backups both belong to reduction.

Reference reading

User-provided SPM course materials: https://mhrumi.github.io/Software-project-management/

Bob Hughes and Mike Cotterell, Software Project Management.

Roger S. Pressman and Bruce R. Maxim, Software Engineering: A Practitioner's Approach.

Barry W. Boehm, Software Engineering Economics.

Steve McConnell, Rapid Development.

ISO 9001:2015 (historical exam context): https://www.iso.org/standard/62085.html

SPM reading index (all lessons): https://mhrumi.github.io/Software-project-management/

Introduction - https://mhrumi.github.io/Software-project-management/01-introduction.html

Project evaluation - https://mhrumi.github.io/Software-project-management/02-project-evaluation.html

Step Wise planning - https://mhrumi.github.io/Software-project-management/03-project-planning.html

Project approach - https://mhrumi.github.io/Software-project-management/04-project-approach.html

Estimation concepts - https://mhrumi.github.io/Software-project-management/05a-estimation-concepts.html

Estimation models - https://mhrumi.github.io/Software-project-management/05b-estimation-models.html

Activity planning - https://mhrumi.github.io/Software-project-management/06-activity-planning.html

Risk management - https://mhrumi.github.io/Software-project-management/07-risk-management.html

Resource allocation - https://mhrumi.github.io/Software-project-management/08-resource-allocation.html

Monitoring and control - https://mhrumi.github.io/Software-project-management/09-monitoring-control.html

Contracts - https://mhrumi.github.io/Software-project-management/10-managing-contracts.html

Managing people - https://mhrumi.github.io/Software-project-management/11-managing-people.html

Teams and leadership - https://mhrumi.github.io/Software-project-management/12-working-in-teams.html

Software quality - https://mhrumi.github.io/Software-project-management/13-software-quality.html

These are supporting reading references, not claims that this paper or its answer key appears in those books. Course-specific terminology and source ambiguities are noted beside the affected

answers.

Shahjalal University of Science and Technology
Department of Computer Science and Engineering
4th Year 1st Semester Final Examination (Session: 2020-21)
Course Code: CSE 421, **Credits:** 2 **Course Title:** Software Project Management
Time: 2 hrs **Total Marks:** 50 **Group A**

[Answer all the questions]

1) ***Answer any five(5) questions from the following:***

a) Define Internal Stakeholders in the context of Software Project Management (SPM) with suitable examples. 1

b) Define Generic Software and illustrate with appropriate examples. 1

c) Explain the term "Code-Like-Hell Programming" as a classical mistake in SPM. 1

d) Explain the term "Short-Changed Upstream Activities" as a classical mistake in SPM. 1

e) Define project scope in Software Project Management with an example. 1

f) Explain the PDCA Loop with an example in software development. 1

g) Write the full form of SMART in the context of identifying project objectives. 1

2) ***Answer any four (4) questions from the following:***

a) Define the term *Software Crisis*. What were the underlying reasons behind it? Give one suitable example. 2.5

b) Project Management consists of five process groups: Initiating, Planning, Executing, Monitoring and Controlling and Closing. Explain each with an example. 2.5

c) What is a *CI/CD Pipeline*? Explain how it works in modern software development. 2.5

d) Distinguish between *risk* and *opportunity* using both real-life and software engineering examples. 2.5

e) Explain the concepts of *Person-Month*, *Brooks' Law*, and *Cost Drivers* in relation to the *COCOMO model*. 2.5

f) Suppose you are planning to develop a video platforming system like YouTube. Describe the entire development process of this system in terms of *Software Project Management*. 2.5

3) ***Answer any two (2) questions from the following:***

a) What is the W5HH principle in Software Project Management? Explain it with suitable examples. 5

b) Industry "Messi" is developing a payroll system for a small company. This is considered a small and simple project.
Industry "CR7" is developing a flight control system for the SU-57 air fighter plane, which is a very complex, real-time, and hardware-dependent project.Using the **Basic COCOMO model** and the given constant table, analyze and compare the following for both projects (assume KLOC = 40): 5

- Effort Estimation (Person-Months)
- Development Time (Schedule in Months)
- Number of People required

Constants:

Mode	a	b	c	d
Organic	2	1	2	1
Semi-detached	3	2	2	1
Embedded	4	3	2	1

CamScanner

c) At the start of the COVID-19 pandemic, Slack's server could not handle the surge in remote work traffic. Business worldwide faced disruptions when Slack went down multiple times. The issue highlighted lack of preparation for demand spikes. 5
Which type of risk is this and why? Explain how can you minimize this type of risk.

Group B

Answer any five(5) questions from the following:

4)

a) Define Continuous Process Improvement(CPI) in the context of software projects. 1

b) Define MoSCoW Prioritization technique with examples. 1

c) Write any two reasons why Risk Management is important in software projects. 1

d) Explain the Blended Approach for risk management. 1

e) State the purpose of Jenkins Software in software development. 1

f) Define Software Process Improvement (SPI) and explain its purpose. 1

g) Define Risk Mitigation in software project management. 1

5) ***Answer any four (4) questions from the following:***

a) Write down the key clauses of ISO 9001. 2.5

b) Why is Work Breakdown Structure (WBS) important when defining project scope? 2.5

c) Describe the role of a Project Manager in defining project planning and project scope with example. 2.5

d) Differentiate between proactive and reactive strategies for risk management with suitable examples. 2.5

e) How is Software Project Management differ from traditional Project Management? 2.5

f) Discuss the major factors that cause software project failure. 2.5

6) ***Answer any two (2) questions from the following:***

a) Analyze the different levels of a Work Breakdown Structure (WBS) for developing an E-Commerce website. Explain the key components included at each level. 5

b) A software firm has standardized processes. Every team follows the same SDLC phases (requirements, design, coding, testing, deployment) with documented checklists. Training sessions are given to all new employees. 5
At which CMM level is this company? Explain all the maturity levels from this identified maturity level to the highest maturity level illustrating the key characteristics of maturity levels

c) A company worries about losing customer data. It reduces likelihood by using reliable databases, reduces impact with daily backups, transfers risk with cyber-insurance, and accepts small risks of minor data corruption. - 5

Explain four Risk Mitigation Strategies with example according to mentioned scenario.

CamScanner